AI & Python Summer Camp

Instructor Manual

Original Development: New Jersey Institute of Technology (NJIT)

Based on Summer 2025 Workshop

Version 1.0 — March 8, 2026

Contents

1 Introduction **4**
1.1 Who This Manual Is For . . . 4
1.2 What Students Will Learn . . . 4
1.3 Curriculum Overview . . . 4
1.4 Required Software . . . 5

2 Week 1: Python Fundamentals **6**
2.1 Week 1 Overview . . . 6
2.2 Day 1A: Introduction to Python . . . 6
2.2.1 Learning Objectives . . . 6
2.2.2 Lesson: What is Programming? (30 min) . . . 6
2.2.3 Lesson: Hello World & Variables . . . 7
2.2.4 Lesson: Arithmetic Operations . . . 7
2.2.5 Student Exercise . . . 7
2.3 Day 1B: Control Flow . . . 7
2.3.1 Learning Objectives . . . 7
2.3.2 Lesson: Conditionals (45 min) . . . 8
2.3.3 Lesson: Lists (30 min) . . . 8
2.3.4 Lesson: Loops (45 min) . . . 8
2.3.5 Student Exercise . . . 8
2.4 Day 1C: Data Visualization . . . 9
2.4.1 Learning Objectives . . . 9
2.4.2 Lesson: Introduction to Libraries (20 min) . . . 9
2.4.3 Lesson: Reading CSV Data (40 min) . . . 9
2.4.4 Lesson: Creating Visualizations (60 min) . . . 9
2.4.5 Student Exercise . . . 10

3 Week 2: Object-Oriented Programming & PyGame **11**
3.1 Week 2 Overview . . . 11
3.2 Day 2A: Functions & Object-Oriented Programming . . . 11
3.2.1 Learning Objectives . . . 11
3.2.2 Lesson: Functions (45 min) . . . 11
3.2.3 Lesson: Introduction to OOP (60 min) . . . 12
3.2.4 Student Exercise . . . 12
3.3 Day 2B: Introduction to PyGame . . . 13
3.3.1 Learning Objectives . . . 13
3.3.2 Lesson: PyGame Basics (45 min) . . . 13
3.3.3 Lesson: Handling Input (45 min) . . . 13
3.3.4 Student Exercise . . . 14
3.4 Day 2C: Building the Robot Simulation . . . 14
3.4.1 Learning Objectives . . . 14

3.4.2 Lesson: The Robot Class (60 min) . . . 14
3.4.3 Lesson: Obstacle Class (30 min) . . . 15
3.4.4 Lesson: Putting It Together (30 min) . . . 15
3.4.5 Student Exercise . . . 16

4 Week 3: Genetic Algorithms 17
4.1 Week 3 Overview . . . 17
4.1.1 What Are Genetic Algorithms? . . . 17
4.2 Day 3A: String Evolution . . . 18
4.2.1 Learning Objectives . . . 18
4.2.2 The Problem . . . 18
4.2.3 Lesson: The Phrase Class, Crossover, and Mutation (80 min) . . . 18
4.2.4 Lesson: The Genetic Algorithm (45 min) . . . 19
4.2.5 Student Exercise . . . 19
4.3 Day 3B-C: Robot Navigation with GAs . . . 20
4.3.1 Learning Objectives . . . 20
4.3.2 Encoding Robot Behavior . . . 20
4.3.3 Fitness Function . . . 20
4.3.4 Student Exercise . . . 21

5 Week 4: Neural Networks 22
5.1 Week 4 Overview . . . 22
5.2 Day 4A: Neural Network Concepts . . . 22
5.2.1 Learning Objectives . . . 22
5.2.2 What Is a Neural Network? . . . 22
5.2.3 How a Neuron Works . . . 23
5.2.4 How Training Works . . . 23
5.3 Day 4B: Building Neural Networks with PyTorch . . . 23
5.3.1 Learning Objectives . . . 23
5.3.2 Lesson: PyTorch Basics (30 min) . . . 23
5.3.3 Lesson: Building a Neural Network (45 min) . . . 24
5.3.4 Lesson: Loading MNIST (30 min) . . . 24
5.4 Day 4C: Training and Evaluation . . . 25
5.4.1 Learning Objectives . . . 25
5.4.2 Lesson: Training Loop (60 min) . . . 25
5.4.3 Lesson: Evaluation (30 min) . . . 25
5.4.4 Lesson: Visualizing Predictions (30 min) . . . 26
5.4.5 Student Exercise . . . 26

6 Week 5: Integration & Capstone Project 27
6.1 Week 5 Overview . . . 27
6.2 Day 5A: Neural Network Robot Control . . . 27
6.2.1 Learning Objectives . . . 27
6.2.2 Robot Brain Architecture . . . 27
6.2.3 Robot Sensors . . . 28
6.2.4 Integration . . . 28
6.2.5 Training with GA . . . 28
6.3 Day 5B: A* Pathfinding Algorithm . . . 29
6.3.1 Learning Objectives . . . 29
6.3.2 How A* Works . . . 29
6.3.3 A* Implementation . . . 29
6.3.4 Student Exercise . . . 30

6.4 Day 5C: Capstone Projects . . . 30
6.4.1 Project Options . . . 30
6.4.2 Presentation Guidelines . . . 31

7 Setup Guide **32**
7.1 Windows Installation . . . 32
7.1.1 Python . . . 32
7.1.2 VS Code . . . 32
7.1.3 Packages . . . 32
7.2 macOS Installation . . . 32
7.3 Test Installation . . . 32

8 Troubleshooting **34**
8.1 Python Issues . . . 34
8.2 Package Issues . . . 34
8.3 PyGame Issues . . . 34
8.4 PyTorch Issues . . . 34

9 Assessment Rubrics **35**
9.1 Weekly Project Rubrics . . . 35
9.2 Capstone Project Rubric . . . 35

10 Implementation Notes and Challenges Faced **36**
10.1 Challenges Faced During the Camp . . . 36
10.2 Lessons Learned for Future Instructors . . . 37

11 Resources **38**
11.1 Online Resources . . . 38
11.2 Workshop Materials . . . 38

Chapter 1

Introduction

This manual provides a complete curriculum for running a five-week AI and Python summer camp for high school students (ages 15–17) with no prior programming experience.

1.1 Who This Manual Is For

- **University TAs and Instructors** running similar workshops
- **Educators** who want to introduce AI to young learners
- **Self-learners** following the curriculum independently

1.2 What Students Will Learn

By the end of this camp, students will be able to:

1. Write Python programs from scratch
2. Understand object-oriented programming
3. Build interactive simulations using PyGame
4. Implement genetic algorithms for optimization
5. Train neural networks using PyTorch
6. Apply pathfinding algorithms (A*)
7. Complete a capstone project combining multiple AI techniques

1.3 Curriculum Overview

Week	Topic	Project
Week 1	Python fundamentals	Data visualization
Week 2	OOP & PyGame	Robot simulation
Week 3	Genetic algorithms	Evolving robot navigation
Week 4	Neural networks	MNIST digit classifier
Week 5	Integration & Capstone	AI-controlled robot

1.4 Required Software

- **Python 3.10+** — https://python.org
- **VS Code** — https://code.visualstudio.com
- **Python packages:**

```
pip install numpy pandas matplotlib pygame torch torchvision
```

Chapter 2

Week 1: Python Fundamentals

2.1 Week 1 Overview

Learning Objectives:

- Understand what programming is
- Write and run Python programs
- Use variables, data types, and operators
- Work with lists, loops, and conditionals
- Read CSV files and create visualizations of the provided datasheet.

Day	Topic	Files
Day A	Python basics	None (start from scratch)
Day B	Control flow	Practice exercises
Day C	Data visualization	`example.csv`, `reference.py`

2.2 Day 1A: Introduction to Python

2.2.1 Learning Objectives

- Understand what programming is and why we use Python
- Write their first program ("Hello, World!")
- Use variables and basic data types
- Perform arithmetic operations

2.2.2 Lesson: What is Programming? (30 min)

Start by asking: "What is programming?"

Key points:

- Programming is giving instructions to a computer
- Python is popular because it's readable and versatile
- Used in AI, web development, data science, and more

Tip: Show real examples—Simple animation, Instagram's backend, Netflix recommendations, NASA data analysis. This helps students see practical value.

2.2.3 Lesson: Hello World & Variables

```python
# My first Python program
print("Hello, World!")

# Variables store values
name = "Alice"
age = 16
height = 5.6
is_student = True

print(f"My name is {name}")
print(f"I am {age} years old")
print(f"I am {height} feet tall")
```

Key concepts:

- `print()` displays output
- Variables store values
- Data types: strings (`"text"`), integers (`16`), floats (`5.6`), booleans (`True/False`)
- f-strings let you embed variables in text

2.2.4 Lesson: Arithmetic Operations

```python
a = 10
b = 3

print(a + b)    # Addition: 13
print(a - b)    # Subtraction: 7
print(a * b)    # Multiplication: 30
print(a / b)    # Division: 3.333...
print(a // b)   # Integer division: 3
print(a % b)    # Modulo (remainder): 1
print(a ** b)   # Exponent: 1000
```

2.2.5 Student Exercise

Task: Create a program that calculates:

1. Your age in days (age * 365)
2. Your age in hours
3. Your age in minutes
4. **Challenge:** Ask the user for their birth year using `input()`

2.3 Day 1B: Control Flow

2.3.1 Learning Objectives

- Use if/elif/else statements
- Work with lists
- Use for and while loops
- Combine conditions with and/or

2.3.2 Lesson: Conditionals (45 min)

```python
age = 16

if age >= 18:
    print("You can vote!")
elif age >= 16:
    print("You can drive!")
else:
    print("You're still young!")

# Combining conditions
has_license = True
if age >= 16 and has_license:
    print("You can drive legally")
```

2.3.3 Lesson: Lists (30 min)

```python
# Creating lists
fruits = ["apple", "banana", "cherry"]
numbers = [1, 2, 3, 4, 5]

# Accessing elements (0-indexed)
print(fruits[0])  # apple
print(fruits[-1]) # cherry (last element)

# Modifying lists
fruits.append("orange")  # Add to end
fruits.remove("banana")  # Remove by value
print(len(fruits))       # Length: 3
```

2.3.4 Lesson: Loops (45 min)

```python
# For loop - iterate over a list
fruits = ["apple", "banana", "cherry"]
for fruit in fruits:
    print(f"I like {fruit}")

# For loop with range
for i in range(5):
    print(i)  # 0, 1, 2, 3, 4

# While loop
count = 0
while count < 5:
    print(count)
    count += 1
```

2.3.5 Student Exercise

Task: Write a number guessing game:

1. Pick a secret number between 1 and 100
2. Let user guess until they get it right
3. Tell them "higher" or "lower" after each guess
4. Count and display total guesses at the end

2.4 Day 1C: Data Visualization

2.4.1 Learning Objectives

- Import and use libraries
- Read CSV files with pandas
- Create plots with matplotlib
- Customize plot appearance

2.4.2 Lesson: Introduction to Libraries (20 min)

```python
# Libraries extend Python's capabilities
import math
print(math.sqrt(16))  # 4.0
print(math.pi)        # 3.14159...

# Import with alias
import pandas as pd
import matplotlib.pyplot as plt
```

2.4.3 Lesson: Reading CSV Data (40 min)

```python
import pandas as pd

# Read a CSV file
data = pd.read_csv("example.csv")

# Explore the data
print(data.head())        # First 5 rows
print(data.shape)         # (rows, columns)
print(data.columns)       # Column names
print(data.describe())    # Statistics

# Access specific columns
print(data["column_name"])
print(data["column_name"].mean())
```

2.4.4 Lesson: Creating Visualizations (60 min)

```python
import matplotlib.pyplot as plt
import pandas as pd

data = pd.read_csv("example.csv")

# Line plot
plt.figure(figsize=(10, 6))
plt.plot(data["x_column"], data["y_column"])
plt.xlabel("X Axis Label")
plt.ylabel("Y Axis Label")
plt.title("My First Plot")
plt.grid(True)
plt.show()

# Bar chart
plt.figure(figsize=(10, 6))
plt.bar(data["categories"], data["values"])
plt.xlabel("Category")
```

```
plt.ylabel("Value")
plt.title("Bar Chart Example")
plt.show()

# Scatter plot
plt.figure(figsize=(10, 6))
plt.scatter(data["x"], data["y"], c="blue", alpha=0.5)
plt.xlabel("X")
plt.ylabel("Y")
plt.title("Scatter Plot")
plt.show()
```

2.4.5 Student Exercise

Task: Create a visualization of stock price data.

1. Load the provided `example.csv` file
2. Create a line plot showing price over time
3. Add appropriate labels and title
4. **Challenge:** Add a second y-axis for trading volume

Chapter 3

Week 2: Object-Oriented Programming & PyGame

3.1 Week 2 Overview

Learning Objectives:

- Understand functions and why we use them
- Learn classes and objects (OOP)
- Use PyGame to create graphics
- Build a robot simulation

Day	Topic	Files
Day A	Functions & intro to OOP	Practice exercises
Day B	PyGame basics	`main.py` template
Day C	Robot simulation	`robot.py`, `obstacle.py`

3.2 Day 2A: Functions & Object-Oriented Programming

3.2.1 Learning Objectives

- Write reusable functions
- Understand parameters and return values
- Learn what classes and objects are
- Create simple classes

3.2.2 Lesson: Functions (45 min)

```python
# Basic function
def greet(name):
    print(f"Hello, {name}!")

greet("Alice")
greet("Bob")

# Function with return value
def add(a, b):
```

```
    return a + b

result = add(5, 3)
print(result)  # 8

# Default parameters
def greet(name, greeting="Hello"):
    print(f"{greeting}, {name}!")

greet("Alice")              # Hello, Alice!
greet("Bob", "Hi")          # Hi, Bob!
```

3.2.3 Lesson: Introduction to OOP (60 min)

Instructor tip: Use the "blueprint" analogy—a class is like a blueprint for a house. The blueprint defines what a house has (rooms, doors), but isn't a house itself. When you build from it, you create an actual house (object).

```
class Dog:
    def __init__(self, name, age):
        self.name = name  # Attribute
        self.age = age

    def bark(self):  # Method
        print(f"{self.name} says Woof!")

    def describe(self):
        print(f"{self.name} is {self.age} years old")

# Create objects (instances)
dog1 = Dog("Buddy", 3)
dog2 = Dog("Max", 5)

dog1.bark()       # Buddy says Woof!
dog2.describe()   # Max is 5 years old
```

Key concepts:

- **Class:** A blueprint/template
- **Object:** An instance created from a class
- **Attributes:** Variables belonging to an object (`self.name`)
- **Methods:** Functions belonging to a class
- **__init__:** Constructor, runs when object is created

3.2.4 Student Exercise

Task: Create a `Student` class with:

1. Attributes: name, age, grades (list)
2. Method: `add_grade(grade)` to add a grade
3. Method: `get_average()` to calculate average grade
4. Create 3 student objects and test them

3.3 Day 2B: Introduction to PyGame

3.3.1 Learning Objectives

- Set up a PyGame window
- Understand the game loop
- Draw shapes and handle colors
- Respond to keyboard/mouse input

3.3.2 Lesson: PyGame Basics (45 min)

```python
import pygame

# Initialize PyGame
pygame.init()

# Create window
WIDTH, HEIGHT = 800, 600
screen = pygame.display.set_mode((WIDTH, HEIGHT))
pygame.display.set_caption("My First Game")

# Colors (R, G, B)
WHITE = (255, 255, 255)
RED = (255, 0, 0)
BLUE = (0, 0, 255)

# Game loop
running = True
while running:
    # Handle events
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            running = False

    # Fill background
    screen.fill(WHITE)

    # Draw shapes
    pygame.draw.circle(screen, BLUE, (400, 300), 50)
    pygame.draw.rect(screen, RED, (100, 100, 200, 150))

    # Update display
    pygame.display.flip()

pygame.quit()
```

3.3.3 Lesson: Handling Input (45 min)

```python
import pygame

pygame.init()
screen = pygame.display.set_mode((800, 600))

# Position
x, y = 400, 300
speed = 5

running = True
```

```
while running:
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            running = False

    # Get pressed keys
    keys = pygame.key.get_pressed()
    if keys[pygame.K_LEFT]:
        x -= speed
    if keys[pygame.K_RIGHT]:
        x += speed
    if keys[pygame.K_UP]:
        y -= speed
    if keys[pygame.K_DOWN]:
        y += speed

    screen.fill((255, 255, 255))
    pygame.draw.circle(screen, (0, 0, 255), (x, y), 20)
    pygame.display.flip()

    pygame.time.Clock().tick(60)  # 60 FPS

pygame.quit()
```

3.3.4 Student Exercise

Task: Create a simple drawing program:

1. Circle follows the mouse cursor
2. Click to place permanent circles
3. Press 'C' to clear the screen
4. **Challenge:** Add color selection with number keys

3.4 Day 2C: Building the Robot Simulation

3.4.1 Learning Objectives

- Create a Robot class for PyGame
- Implement movement with heading and speed
- Add obstacles to the simulation
- Implement basic collision detection

3.4.2 Lesson: The Robot Class (60 min)

```
import pygame
import math

class Robot:
    def __init__(self, x, y, radius=20):
        self.x = x
        self.y = y
        self.radius = radius
        self.heading = 0  # Angle in degrees
        self.speed = 0
```

```
        self.color = (0, 0, 255)

    def update(self):
        # Convert heading to radians for math
        rad = math.radians(self.heading)
        # Update position based on speed and heading
        self.x += self.speed * math.cos(rad)
        self.y += self.speed * math.sin(rad)

    def draw(self, screen):
        # Draw robot body
        pygame.draw.circle(
            screen,
            self.color,
            (int(self.x), int(self.y)),
            self.radius
        )

        # Draw direction indicator
        rad = math.radians(self.heading)
        end_x = self.x + self.radius * math.cos(rad)
        end_y = self.y + self.radius * math.sin(rad)

        pygame.draw.line(
            screen,
            (255, 0, 0),
            (int(self.x), int(self.y)),
            (int(end_x), int(end_y)),
            3
        )

    def handle_input(self, keys):
        if keys[pygame.K_LEFT]:
            self.heading -= 5
        if keys[pygame.K_RIGHT]:
            self.heading += 5
        if keys[pygame.K_UP]:
            self.speed = min(self.speed + 0.2, 5)
        if keys[pygame.K_DOWN]:
            self.speed = max(self.speed - 0.2, 0)
```

3.4.3 Lesson: Obstacle Class (30 min)

```
class Obstacle:
    def __init__(self, x, y, width, height):
        self.x = x
        self.y = y
        self.width = width
        self.height = height
        self.color = (100, 100, 100)

    def draw(self, screen):
        pygame.draw.rect(screen, self.color,
                         (self.x, self.y, self.width, self.height))

    def contains_point(self, px, py):
        return (self.x <= px <= self.x + self.width and
                self.y <= py <= self.y + self.height)
```

3.4.4 Lesson: Putting It Together (30 min)

```python
import pygame
from robot import Robot
from obstacle import Obstacle

pygame.init()
screen = pygame.display.set_mode((800, 600))

robot = Robot(400, 300)
obstacles = [
    Obstacle(100, 100, 100, 50),
    Obstacle(500, 400, 150, 80),
]

running = True
clock = pygame.time.Clock()

while running:
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            running = False

    keys = pygame.key.get_pressed()
    robot.handle_input(keys)
    robot.update()

    screen.fill((255, 255, 255))
    for obs in obstacles:
        obs.draw(screen)
    robot.draw(screen)
    pygame.display.flip()
    clock.tick(60)

pygame.quit()
```

3.4.5 Student Exercise

Task: Enhance the robot simulation:

1. Add boundary detection (robot stays on screen)
2. Add collision detection with obstacles
3. Display robot's speed and heading on screen
4. **Challenge:** Add multiple robots

Chapter 4

Week 3: Genetic Algorithms

4.1 Week 3 Overview

Learning Objectives:

- Understand how evolution works
- Explain genetic algorithm components
- Implement selection, crossover, and mutation
- Apply GAs to string evolution
- Apply GAs to robot navigation

Day	Topic	Files
Day A	GA concepts & string evolution	`phrase.py`, `genetic.py`
Day B	Robot DNA	`robot_dna.py`
Day C	Evolving navigation	Complete GA robot project

4.1.1 What Are Genetic Algorithms?

Genetic algorithms are optimization techniques inspired by natural evolution:

1. Create a **population** of random solutions
2. Evaluate each solution's **fitness**
3. **Select** the best solutions as parents
4. Create new solutions via **crossover**
5. Introduce random **mutations**
6. Repeat until solved

Tip: Use the "breeding dogs" analogy—if you want faster dogs, select the fastest and breed them. Over generations, dogs get faster. GAs do this with programs.

4.2 Day 3A: String Evolution

4.2.1 Learning Objectives

- Understand fitness functions
- Implement the `Phrase` class
- Implement crossover and mutation
- Watch evolution happen in real-time

4.2.2 The Problem

Evolve random strings to match: **"This is NJIT AI Workshop!"**

4.2.3 Lesson: The Phrase Class, Crossover, and Mutation (80 min)

```python
import random
import string

class Phrase:
    def __init__(self, target, genes=None):
        self.target = target
        if genes is None:
            # Create random genes
            self.genes = ''.join(random.choices(
                string.ascii_letters + string.digits + ' !',
                k=len(target)
            ))
        else:
            self.genes = genes
        self.fitness = self.calculate_fitness()

    def calculate_fitness(self):
        """Count matching characters"""
        score = 0
        for i, char in enumerate(self.genes):
            if char == self.target[i]:
                score += 1
        return score / len(self.target)

    def crossover(self, partner):
        """Combine genes from two parents"""
        child_genes = ""
        midpoint = random.randint(0, len(self.genes))

        for i in range(len(self.genes)):
            if i < midpoint:
                child_genes += self.genes[i]
            else:
                child_genes += partner.genes[i]

        return Phrase(self.target, child_genes)

    def mutate(self, mutation_rate):
        """Randomly change some genes"""
        new_genes = ""
        for char in self.genes:
            if random.random() < mutation_rate:
                new_genes += random.choice(
                    string.ascii_letters + string.digits + ' !'
```

```
                )
            else:
                new_genes += char
        self.genes = new_genes
        self.fitness = self.calculate_fitness()
```

4.2.4 Lesson: The Genetic Algorithm (45 min)

```
class GeneticAlgorithm:
    def __init__(self, target, pop_size=200, mutation_rate=0.01):
        self.target = target
        self.pop_size = pop_size
        self.mutation_rate = mutation_rate
        self.population = [Phrase(target) for _ in range(pop_size)]
        self.generation = 0

    def natural_selection(self):
        """Create mating pool based on fitness"""
        self.mating_pool = []
        max_fitness = max(p.fitness for p in self.population)

        # Avoid division by zero if all fitness values are zero
        if max_fitness == 0:
            max_fitness = 1

        for phrase in self.population:
            # Higher fitness = more copies in mating pool
            n = max(1, int((phrase.fitness / max_fitness) * 100))
            self.mating_pool.extend([phrase] * n)

    def generate(self):
        """Create next generation"""
        new_pop = []
        for _ in range(self.pop_size):
            parent_a = random.choice(self.mating_pool)
            parent_b = random.choice(self.mating_pool)
            child = parent_a.crossover(parent_b)
            child.mutate(self.mutation_rate)
            new_pop.append(child)

        self.population = new_pop
        self.generation += 1

    def get_best(self):
        return max(self.population, key=lambda p: p.fitness)

    def run(self):
        while self.get_best().fitness < 1.0:
            self.natural_selection()
            self.generate()
            best = self.get_best()
            print(f"Gen {self.generation}: {best.genes} ({best.fitness:.1%})")
```

4.2.5 Student Exercise

1. Change target to your name or custom message
2. Try different population sizes (50, 100, 500)
3. Try different mutation rates (0.001, 0.01, 0.1)
4. **Challenge:** Plot fitness over generations with matplotlib

4.3 Day 3B-C: Robot Navigation with GAs

4.3.1 Learning Objectives

- Encode robot behaviors as genes
- Design fitness functions for navigation
- Evolve robots that reach goals
- Visualize evolution in PyGame

4.3.2 Encoding Robot Behavior

Each robot's DNA is a list of movement commands:

```python
class RobotDNA:
    def __init__(self, num_moves=100, genes=None):
        self.num_moves = num_moves
        if genes is None:
            # Random movements: (angle_change, speed)
            self.genes = [
                (random.uniform(-30, 30), random.uniform(1, 5))
                for _ in range(num_moves)
            ]
        else:
            self.genes = genes

    def crossover(self, partner):
        child_genes = []
        midpoint = random.randint(0, len(self.genes))
        for i in range(len(self.genes)):
            if i < midpoint:
                child_genes.append(self.genes[i])
            else:
                child_genes.append(partner.genes[i])
        return RobotDNA(self.num_moves, child_genes)

    def mutate(self, mutation_rate):
        for i in range(len(self.genes)):
            if random.random() < mutation_rate:
                self.genes[i] = (
                    random.uniform(-30, 30),
                    random.uniform(1, 5)
                )
```

4.3.3 Fitness Function

```python
def calculate_fitness(robot, goal, obstacles):
    # Distance to goal (closer = better)
    distance = math.sqrt((robot.x - goal.x)**2 +
                         (robot.y - goal.y)**2)

    # Penalties and bonuses
    collision_penalty = 100 if robot.collided else 0
    goal_bonus = 1000 if distance < goal.radius else 0

    fitness = 1000 - distance - collision_penalty + goal_bonus
    return max(0, fitness)
```

4.3.4 Student Exercise

1. Run the GA robot simulation
2. Experiment with different fitness functions
3. Try adding more obstacles
4. **Challenge:** Evolve robots for multiple goals

Chapter 5

Week 4: Neural Networks

5.1 Week 4 Overview

Learning Objectives:

- Understand what neural networks are
- Learn about neurons, layers, and weights
- Build neural networks with PyTorch
- Train a model on MNIST dataset
- Evaluate model performance

Day	Topic	Files
Day A	NN concepts	Slides, diagrams
Day B	PyTorch basics	`mnist_template.py`
Day C	Training & evaluation	`mnist_complete.py`

5.2 Day 4A: Neural Network Concepts

5.2.1 Learning Objectives

- Understand what neural networks are
- Learn about neurons, layers, weights, biases
- Understand activation functions
- Learn how training works (conceptually)

5.2.2 What Is a Neural Network?

Tip: Be careful with the "brain" analogy—NNs are inspired by brains but work differently. Focus on: neural networks learn patterns from data.

Key concepts:

- **Neurons:** Units that take inputs, apply weights, produce outputs
- **Layers:** Groups of neurons (input, hidden, output)
- **Weights:** Parameters the network learns

- **Bias:** Additional parameter added to each neuron
- **Activation functions:** Non-linear functions (ReLU, sigmoid)

5.2.3 How a Neuron Works

```
# A single neuron (conceptually)
def neuron(inputs, weights, bias):
    # Weighted sum
    total = 0
    for i in range(len(inputs)):
        total += inputs[i] * weights[i]
    total += bias

    # Activation function (ReLU)
    output = max(0, total)
    return output
```

5.2.4 How Training Works

1. **Forward pass:** Input goes through network, produces prediction
2. **Loss calculation:** Compare prediction to correct answer
3. **Backward pass:** Calculate how much each weight contributed to error
4. **Update weights:** Adjust weights to reduce error
5. **Repeat:** Do this for many examples

5.3 Day 4B: Building Neural Networks with PyTorch

5.3.1 Learning Objectives

- Install and import PyTorch
- Create tensors and basic operations
- Build a neural network class
- Load the MNIST dataset

5.3.2 Lesson: PyTorch Basics (30 min)

```
import torch

# Tensors are like numpy arrays
a = torch.tensor([1, 2, 3])
b = torch.tensor([4, 5, 6])

print(a + b)  # tensor([5, 7, 9])
print(a * b)  # tensor([4, 10, 18])

# Create tensors
zeros = torch.zeros(3, 3)
ones = torch.ones(2, 4)
random = torch.randn(3, 3)  # Random normal
```

5.3.3 Lesson: Building a Neural Network (45 min)

```python
import torch
import torch.nn as nn

class SimpleNN(nn.Module):
    def __init__(self):
        super(SimpleNN, self).__init__()
        # Input: 784 pixels (28x28 image flattened)
        # Hidden layer 1: 128 neurons
        # Hidden layer 2: 64 neurons
        # Output: 10 classes (digits 0-9)
        self.fc1 = nn.Linear(784, 128)
        self.fc2 = nn.Linear(128, 64)
        self.fc3 = nn.Linear(64, 10)
        self.relu = nn.ReLU()

    def forward(self, x):
        # Flatten the image
        x = x.view(-1, 784)
        # Pass through layers
        x = self.relu(self.fc1(x))
        x = self.relu(self.fc2(x))
        x = self.fc3(x)
        return x

model = SimpleNN()
print(model)
```

5.3.4 Lesson: Loading MNIST (30 min)

```python
from torchvision import datasets, transforms
from torch.utils.data import DataLoader

# Transform: convert to tensor and normalize
transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.1307,), (0.3081,))
])

# Download and load training data
train_data = datasets.MNIST(
    './data',
    train=True,
    download=True,
    transform=transform
)

test_data = datasets.MNIST(
    './data',
    train=False,
    download=True,
    transform=transform
)

# Create data loaders
train_loader = DataLoader(train_data, batch_size=64, shuffle=True)
test_loader = DataLoader(test_data, batch_size=64, shuffle=False)

print(f"Training samples: {len(train_data)}")
print(f"Test samples: {len(test_data)}")
```

5.4 Day 4C: Training and Evaluation

5.4.1 Learning Objectives

- Set up loss function and optimizer
- Write the training loop
- Evaluate model accuracy
- Visualize predictions

5.4.2 Lesson: Training Loop (60 min)

```python
import torch.optim as optim

# Setup
model = SimpleNN()
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=0.001)

# Training loop
num_epochs = 5
for epoch in range(num_epochs):
    model.train()
    total_loss = 0

    for batch_idx, (data, target) in enumerate(train_loader):
        # Zero gradients
        optimizer.zero_grad()

        # Forward pass
        output = model(data)

        # Calculate loss
        loss = criterion(output, target)

        # Backward pass
        loss.backward()

        # Update weights
        optimizer.step()

        total_loss += loss.item()

        if batch_idx % 200 == 0:
            print(f'Epoch {epoch}, Batch {batch_idx}, Loss: {loss.item():.4f}')

    avg_loss = total_loss / len(train_loader)
    print(f'Epoch {epoch} complete. Average loss: {avg_loss:.4f}')
```

5.4.3 Lesson: Evaluation (30 min)

```python
def evaluate(model, test_loader):
    model.eval()
    correct = 0
    total = 0

    with torch.no_grad():
        for data, target in test_loader:
            output = model(data)
```

```
            _, predicted = torch.max(output, 1)
            total += target.size(0)
            correct += (predicted == target).sum().item()

    accuracy = 100 * correct / total
    print(f'Test Accuracy: {accuracy:.2f}%')
    return accuracy

evaluate(model, test_loader)
```

5.4.4 Lesson: Visualizing Predictions (30 min)

```
import matplotlib.pyplot as plt

def show_predictions(model, test_loader, num_images=10):
    model.eval()
    images, labels = next(iter(test_loader))

    with torch.no_grad():
        outputs = model(images)
        _, predictions = torch.max(outputs, 1)

    fig, axes = plt.subplots(2, 5, figsize=(12, 5))
    for i, ax in enumerate(axes.flat):
        if i < num_images:
            img = images[i].squeeze()
            ax.imshow(img, cmap='gray')
            color = 'green' if predictions[i] == labels[i] else 'red'
            ax.set_title(f'Pred: {predictions[i]}, True: {labels[i]}',
                         color=color)
            ax.axis('off')
    plt.tight_layout()
    plt.show()

show_predictions(model, test_loader)
```

5.4.5 Student Exercise

1. Train the model and achieve >95% accuracy
2. Try different architectures (more/fewer layers)
3. Experiment with learning rates
4. **Challenge:** Save and load the trained model

Chapter 6

Week 5: Integration & Capstone Project

6.1 Week 5 Overview

Learning Objectives:

- Combine neural networks with robot simulation
- Learn about the A* pathfinding algorithm
- Complete a capstone project
- Present final projects

Day	Topic	Files
Day A	NN robot control	`robot_brain.py`
Day B	A* pathfinding	`a_star.py`
Day C	Capstone presentations	Student projects

6.2 Day 5A: Neural Network Robot Control

6.2.1 Learning Objectives

- Design NN architecture for robot control
- Connect sensors to neural network inputs
- Use NN outputs for movement commands
- Train using GA or supervised learning

6.2.2 Robot Brain Architecture

```python
import torch
import torch.nn as nn

class RobotBrain(nn.Module):
    def __init__(self):
        super(RobotBrain, self).__init__()
        # Input: 5 sensor readings
        # Output: 2 values (turn angle, speed)
        self.fc1 = nn.Linear(5, 16)
```

```
        self.fc2 = nn.Linear(16, 8)
        self.fc3 = nn.Linear(8, 2)
        self.relu = nn.ReLU()
        self.tanh = nn.Tanh()

    def forward(self, x):
        x = self.relu(self.fc1(x))
        x = self.relu(self.fc2(x))
        x = self.tanh(self.fc3(x))  # Output between -1 and 1
        return x
```

6.2.3 Robot Sensors

```
def get_sensor_readings(self, obstacles):
    """Cast rays in 5 directions to detect obstacles"""
    sensor_angles = [-45, -22.5, 0, 22.5, 45]
    readings = []

    for angle_offset in sensor_angles:
        angle = self.heading + angle_offset
        distance = self.cast_ray(angle, obstacles)
        # Normalize to 0-1 range
        readings.append(distance / 200.0)

    return readings
```

6.2.4 Integration

```
def update_with_nn(robot, brain, obstacles):
    # Get normalized sensor readings
    sensors = robot.get_sensor_readings(obstacles)
    sensor_tensor = torch.FloatTensor(sensors).unsqueeze(0)

    # Get neural network output
    with torch.no_grad():
        output = brain(sensor_tensor)

    # Convert outputs to robot commands
    turn = output[0, 0].item() * 30    # -30 to 30 degrees
    speed = (output[0, 1].item() + 1) * 2.5  # 0 to 5

    robot.heading += turn
    robot.speed = speed
    robot.update()
```

6.2.5 Training with GA

```
def weights_to_genes(model):
    """Convert NN weights to a flat list (genes)"""
    genes = []
    for param in model.parameters():
        genes.extend(param.data.flatten().tolist())
    return genes

def genes_to_weights(model, genes):
    """Set NN weights from genes"""
    idx = 0
    for param in model.parameters():
        size = param.numel()
```

```
        param.data = torch.FloatTensor(
            genes[idx:idx+size]
        ).reshape(param.shape)
        idx += size
```

6.3 Day 5B: A* Pathfinding Algorithm

6.3.1 Learning Objectives

- Understand graph-based pathfinding
- Learn the A* algorithm
- Implement A* in Python
- Visualize pathfinding

6.3.2 How A* Works

A* finds the shortest path by:

1. Maintaining a priority queue of nodes to explore
2. For each node, calculating: f(n) = g(n) + h(n)
    - g(n): Cost from start to current node
    - h(n): Estimated cost from current to goal (heuristic)
3. Always exploring the node with lowest f(n)
4. Stopping when goal is reached

6.3.3 A* Implementation

```
import heapq

def heuristic(a, b):
    """Manhattan distance"""
    return abs(a[0] - b[0]) + abs(a[1] - b[1])

def a_star(grid, start, goal):
    rows, cols = len(grid), len(grid[0])
    open_set = [(0, start)]
    came_from = {}
    g_score = {start: 0}
    f_score = {start: heuristic(start, goal)}

    while open_set:
        _, current = heapq.heappop(open_set)

        if current == goal:
            # Reconstruct path
            path = [current]
            while current in came_from:
                current = came_from[current]
                path.append(current)
            return path[::-1]

        # Check neighbors (up, down, left, right)
```

```
        for dx, dy in [(0,1), (0,-1), (1,0), (-1,0)]:
            neighbor = (current[0]+dx, current[1]+dy)

            # Check bounds and obstacles
            if (0 <= neighbor[0] < rows and
                0 <= neighbor[1] < cols and
                grid[neighbor[0]][neighbor[1]] != 1):

                tentative_g = g_score[current] + 1

                if tentative_g < g_score.get(neighbor, float('inf')):
                    came_from[neighbor] = current
                    g_score[neighbor] = tentative_g
                    f_score[neighbor] = tentative_g + heuristic(neighbor, goal)
                    heapq.heappush(open_set, (f_score[neighbor], neighbor))

    return None  # No path found
```

6.3.4 Student Exercise

1. Implement A* visualization with PyGame
2. Add diagonal movement
3. Compare A* with random walk
4. **Challenge:** Combine A* with robot simulation

6.4 Day 5C: Capstone Projects

6.4.1 Project Options

Students choose one of the following:

Option 1: Smart Robot Navigator

- Robot uses NN to navigate obstacles
- Train using genetic algorithm
- Visualize learning progress

Option 2: Pathfinding Visualizer

- Interactive grid editor
- Compare A*, BFS, and random search
- Show step-by-step exploration

Option 3: Custom Image Classifier

- Train NN on custom dataset (draw your own!)
- Build simple drawing interface
- Real-time prediction

Option 4: Evolution Simulator

- Creatures evolve to survive
- Multiple fitness criteria
- Visualize population over generations

6.4.2 Presentation Guidelines

Each student/team presents (5-10 minutes):

1. What problem did you solve?
2. What approach did you use?
3. Demo your project
4. What did you learn?
5. What would you improve?

Chapter 7

Setup Guide

7.1 Windows Installation

7.1.1 Python

1. Go to https://python.org/downloads
2. Download latest Python 3.x
3. **IMPORTANT:** Check "Add Python to PATH"
4. Verify: open Command Prompt, type `python --version`

7.1.2 VS Code

1. Go to https://code.visualstudio.com
2. Download and install
3. Open VS Code
4. Press Ctrl+Shift+X, search "Python", install Microsoft extension

7.1.3 Packages

Open terminal (Ctrl+‘) and run:

```
pip install numpy pandas matplotlib pygame
pip install torch torchvision --index-urlhttps://download.pytorch.org/whl/cpu
```

Note: We use CPU-only PyTorch to avoid GPU driver issues.

7.2 macOS Installation

1. Download Python from https://python.org/downloads/macos
2. Use `python3` and `pip3` instead of `python` and `pip`

7.3 Test Installation

Create `test_setup.py`:

```python
import sys
print(f"Python: {sys.version}")

import numpy; print(f"NumPy: {numpy.__version__}")
import pandas; print(f"Pandas: {pandas.__version__}")
import matplotlib; print(f"Matplotlib: {matplotlib.__version__}")
import pygame; print(f"Pygame: {pygame.__version__}")
import torch; print(f"PyTorch: {torch.__version__}")

print("\nSetup complete!")
```

Chapter 8

Troubleshooting

8.1 Python Issues

"python is not recognized"

- Python not in PATH
- Fix: Reinstall and check "Add Python to PATH"

"pip is not recognized"

- Use `python -m pip install package_name` instead

8.2 Package Issues

"No module named X"

- Package not installed. Run `pip install X`

"FileNotFoundError" for CSV

- File not in same directory as script
- Check with: `import os; print(os.getcwd())`

8.3 PyGame Issues

Window freezes or black screen

- Missing event loop. Make sure code includes:

```
for event in pygame.event.get():
    if event.type == pygame.QUIT:
        running = False
pygame.display.flip()
```

8.4 PyTorch Issues

CUDA errors

- Use CPU-only PyTorch for this camp
- If issues persist, reduce batch size

Chapter 9

Assessment Rubrics

9.1 Weekly Project Rubrics

Criteria	**4**	**3**	**2**	**1**
Functionality	Complete	Minor issues	Major issues	Not working
Code quality	Excellent	Good	Fair	Poor
Understanding	Expert	Proficient	Developing	Beginning

9.2 Capstone Project Rubric

Criteria	**4**	**3**	**2**	**1**
Technical implementation	Excellent	Good	Fair	Poor
Creativity/originality	Exceptional	Good	Some	Minimal
Presentation	Clear, engaging	Clear	Unclear	Incomplete
Documentation	Complete	Adequate	Minimal	Missing

Chapter 10

Implementation Notes and Challenges Faced

This section summarizes practical difficulties that instructors may encounter while running the camp and offers suggestions for handling them.

10.1 Challenges Faced During the Camp

- **Different student experience levels:** Although the camp was designed for beginners, some students learned concepts much faster than others. This created pacing challenges during coding activities, especially in Weeks 2 through 4.
- **Software setup issues:** A significant portion of early instructional time was spent helping students install Python, VS Code, and required packages. Package installation problems, especially with PyTorch and PyGame, slowed the beginning of some sessions.
- **Debugging frustration:** Many students were enthusiastic at first but became discouraged when their programs did not run because of small syntax or indentation errors. Extra instructor support was often needed to help students stay motivated.
- **Time management during project work:** Open-ended activities, particularly the robot simulation and capstone work, took longer than expected for some groups. Students often needed more time for testing, debugging, and refining their ideas.
- **Balancing theory and hands-on work:** Some students were more engaged during interactive coding portions than during conceptual explanations. Instructors had to frequently switch between explanation, demonstration, and live troubleshooting to keep students focused.
- **Hardware and computer performance differences:** Students used different computers with different performance levels, which occasionally affected package installation, program execution speed, and the smoothness of PyGame or PyTorch exercises.
- **Maintaining engagement over multiple weeks:** As the material became more advanced, especially during the neural network and capstone sections, some students needed additional encouragement and simpler checkpoints to remain confident and involved.
- Indentation errors in Python were the most common debugging issue, particularly during the early weeks when students were first introduced to control flow and loops. Instructors frequently needed to explain how Python uses indentation instead of braces.

10.2 Lessons Learned for Future Instructors

- Build in extra setup time during the first week.
- Prepare backup files and working reference solutions for each lesson.
- Use short milestone-based exercises so students can experience small wins.
- Pair faster students with students who need more help when possible.
- Keep conceptual explanations brief and return quickly to interactive examples.
- Reserve additional buffer time for final project development and presentations.

Chapter 11

Resources

11.1 Online Resources

- **Python:** https://docs.python.org/3/tutorial/
- **PyGame:** https://pygame.org/docs/
- **PyTorch:** https://pytorch.org/tutorials/
- **3Blue1Brown Neural Networks:** Excellent YouTube series
- **A* Visualization:** https://www.redblobgames.com/pathfinding/a-star/

11.2 Workshop Materials

All instructional materials used in this camp are stored in a shared folder accessible to instructors and organizers.

https://drive.google.com/drive/folders/1Wvt64YaM8tT7vonekZzuLimUDyxFoeG7?usp=sharing

The folder contains:

- Student starter code
- Completed reference solutions
- Lecture slides and diagrams
- Example datasets used in exercises

Access to the shared folder is provided to instructors before the start of the camp.